

Implementing Agentic RAG using Spring AI

JPrime 2026, workshop

Jordan Jovkov

In this workshop, we will build step by step a working Agentic RAG using Java, Spring Boot 4, and Spring AI 2. We will start with making a functionality performing a simple direct question to an LLM and evolve the system into a fully functional Agentic RAG capable of autonomously deciding which sources to retrieve information from and how to combine it.

By the end of the workshop, we will have a working system integrating a vector database, a relational database, an external application accessed via a REST client, and an MCP Server as independent data sources.

At the beginning, I will demonstrate the final version of the completed project so the direction what we aim to achieve will be clear. Due to limited time for live coding, sometimes we will write code, and sometimes we will copy pre-prepared code fragments, but in all cases we will review them in detail and follow the logical sequence that would apply in real development process.

Particular attention will be paid to how changes of specific technical parameters and strategies will affect the quality of the final result.

Domain

We will work in the domain of video production hardware - specifically, non-linear video editing cards that were popular in the late 1990s and early 2000s. I have professional experience in this field during that period, working in television and video production companies. I have noticed that there is little information available on the internet about this topic. Most LLMs, with the exception of the largest ones, are not fully familiar with it in detail.

We will follow best practices such as strong separation of concerns and scalability. Therefore, we will start with an environment consisting of a project containing five modules, each corresponding to a separate Java Spring Boot project. Each module is independent of the others and can act as a standalone unit with its own responsibilities.

Three of the modules will be pre-built and will only be used as data sources for creating our Agentic RAG.

agenticrag-workshop

- |— rag-ingestor
- |— rag-retriever
- |— video-production-store
- |— video-production-store-client
- |— video-production-store-mcp-server

What we have at the beginning

As mentioned earlier we have three already prepared projects.

video-production-store

Let us imagine that we have a fully functional application serving as a store/warehouse for selling video editing products. Part of this application manages the descriptions of video editing cards, stock availability, customer information, and data about purchases made.

This is a standard Spring Boot backend application where resource management is handled via a REST approach. The data is stored in a MySQL relational database.

Swagger REST API documentation is available:

<http://localhost:8080/video-production-store/swagger-ui.html>

Domain objects:

VideoEditingCard

- id
- name
- manufacturer
- description
- price

StockAvailability

- id
- VideoEditingCard
- availability

Customer

- id
- name
- email
- phone
- address
- notes

Order

- id
- Customer
- videoEditingCard
- orderDate
- orderNote

Pre-loaded data:

The data is stored in the projects MySQL database. We will only access the data using reading operations for the scope of this workshop.

ID	NAME	MANUFACTURER	DESCRIPTION	PRICE
1	Movie Machine Pro	Fast Multimedia AG	For enthusiasts and professional Productions	350.00
2	DPS Velocity	DPS	Professional for TV and Productions	1200.00
3	Media 100	Data Translation	MAC oriented Professional for TV and Productions	450.00
4	MiroMotion DC30	Pinacle	For enthusiasts and professionals	280.00

Video Editing Cards

ID	VIDEO_EDITING_CARD_ID	AVAILABILITY
1	1	5
2	2	23
3	3	47
4	4	12

Stock Availability

ID	NAME	EMAIL	PHONE	ADDRESS	NOTES
1	Ivan Ivanov	ivan.ivanov@gmail.com	0888111222	Sofia, Vitosha 1	VIP client
2	Petar Petrov	petar.petrov@gmail.com	0888333444	Plovdiv, Maritsa 5	Unable to pick up the phone by 2 pm
3	Georgi Georgiev	georgi.georgiev@gmail.com	0888555666	Varna, Chernomorie 12	He prefers quick delivery
4	Nikolay Nikolaev	nikolay.nikolaev@gmail.com	0888777888	Sofia, Tech Park	The building lift is not working

Customers

ID	CUSTOMER_ID	VIDEO_EDITING_CARD_ID	ORDER_DATE	ORDER_NOTE
1	1	1	2025-01-15	Urgent delivery
2	2	1	2025-02-20	Only credit card payment
3	3	2	2025-03-10	Customer gift
4	4	3	2025-03-25	The same delivery for the next order
5	5	3	2025-04-05	Use light weight packages
6	6	4	2025-04-18	See the invoice details

Orders

Available endpoints:

VIDEO EDITING CARDS

```
GET    /api/video-editing-cards
GET    /api/video-editing-cards/{id}
GET    /api/video-editing-cards/name?name=
GET    /api/video-editing-cards/price?minPrice=&maxPrice=
POST   /api/video-editing-cards
PUT    /api/video-editing-cards/{id}
DELETE /api/video-editing-cards/{id}
```

CUSTOMERS

```
GET    /api/customers
GET    /api/customers/{id}
GET    /api/customers/name?name=
GET    /api/customers/video-card/{videoCardId}
POST   /api/customers
PUT    /api/customers/{id}
DELETE /api/customers/{id}
```

STOCK AVAILABILITIES

```
GET    /api/stock-availabilities
GET    /api/stock-availabilities/{id}
GET    /api/stock-availabilities/video-card/{videoCardId}
GET    /api/stock-availabilities/min-quantity?minQuantity=
POST   /api/stock-availabilities
PUT    /api/stock-availabilities/{id}
DELETE /api/stock-availabilities/{id}
```

ORDERS

```
GET    /api/orders
GET    /api/orders/{id}
GET    /api/orders/customer/{customerId}
POST   /api/orders
PUT    /api/orders/{id}
DELETE /api/orders/{id}
```

SWAGGER

```
GET    /swagger-ui.html
GET    /api-docs
```

For integration of our video-production-store with other applications, two additional modules (Java projects) have been pre-built. They provide integration in two ways:

- via a pre-built REST client, which serves as a library — an external embeddable API
- via a pre-built MCP server, which mirrors the REST functionality of our video production hardware store video-production-store

video-production-store-client

The module provides a pre-built REST client library for the video-production-store application.

It encapsulates all the REST communication logic and can be embedded as a dependency in any other Java project that needs to integrate with video-production-store.

We will use it in the rag-retriever project in Step 6 and internally it is also used by the video-production-store-mcp-server mentioned in Step 7.

```
└─ video-production-store-client [library jar, RestClient]
    └─ video-production-store [port 8080, REST + MySQL]
```

video-production-store-mcp-server

The video-production-store-mcp-server module is a pre-built MCP server that exposes the functionality of the video-production-store application via the MCP protocol.

It provides the same data access capabilities as the REST API, but through a standardized protocol that allows AI agents to discover and invoke its tools autonomously.

Internally, it uses the video-production-store-client library to communicate with the core application.

We will use it in Step 7 as a black-box MCP Server - the agent will interact with it without any knowledge of its internal implementation.

```
└─ video-production-store-mcp-server [port 8083, SSE]
    └─ video-production-store-client [library jar, RestClient]
└─ video-production-store [port 8080, REST + MySQL]
```

— continues on the next page —

What we need to develop

Based on our core application video-production-store, we decided to introduce RAG functionality that will allow us to:

- access technical data and specifications about the various video editing cards
- perform various queries based on our customer database and purchase history, representing different business use cases

To achieve this, we will create two new Java Spring Boot projects as modules corresponding to the two main phases of the RAG process - Ingestion and Retrieval.

We will use the latest versions of Spring Boot 4 and Spring AI 2. The new version of Spring Boot 4 slightly changes the concept of dependency granularity, and we will keep this in mind when using certain libraries.

Infrastructure Prerequisites

- Java 25 installed.
- Maven installed.
- Docker Desktop installed.
- Create a Groq account to use an appropriate LLM via the Groq free tier.
- We will copy a pre-generated OpenAI API key provided by the instructor to use a quality embedding model. Costs are covered by the instructor. Participants who already have their own OpenAI developer account may use it if they prefer.
- Create a system environment variable named GROQ_API_KEY with the value of the newly created Groq API key.
- If not already set, create a system environment variable named OPENAI_API_KEY with the provided key value.
- In a suitable directory, pull the remote repository with the prepared project structure from the provided GitHub url:
 - <https://github.com/jordanjovkov/jprime-agenticrag-workshop>

— continues on the next page —

The two projects we will work on:

rag-ingestor

The rag-ingestor module will provide functionality for reloading external data into our system.

The technical data will be extracted from official technical documentation and user manuals of these video editing cards. This information is contained in PDF documents stored at a specific location on the file system.

The Knowledge Base documents will be PDF files for the following video editing cards: *Fast Movie Machine Pro*, *DPS Velocity*, *Media 100* and *Miro Motion 30*.

Technology stack

- Java Spring Boot
- REST, port 8081
- Spring AI
- Tika Document Reader library for reading file-based documents
- pgvector vector database
- Open Telemetry + Grafana
- Docker infrastructure

For the embedding model we will use OpenAI's paid *text-embedding-3-small*.

This embedding model is paid but we chose it having in mind its high quality. A pre-generated OpenAI API key is provided by the instructor and everyone can use it.

Expected output

- REST endpoint for reloading the Knowledge Base
- REST endpoint for checking whether data exists in the vector database for a given search query - will be copied from the pre-built project, as it is a supporting rather than a core function

Once we have created rag-ingestor, verified that it works, and confirmed that our Knowledge Base is loaded with domain-specific data, we will move on to developing and incrementally evolving the rag-retriever project, through which we will actually use our RAG.

— continues on the next page —

The two projects we will work on:

rag-retriever

The rag-retriever module will provide the following functionality:

- direct communication with the LLM
- retrieval of technical information about our products in the store
- performing various types of queries related to video editing cards, customers, and orders already placed
- the output of these queries can be in various formats — from informational text to structured objects containing specific required data

This is the core project of the workshop. It will evolve incrementally through all development steps - starting as a simple LLM client, then becoming a Naive RAG, and finally evolving into a fully functional Agentic RAG capable of routing queries across multiple data sources. Each step builds directly on the previous one.

We will use the chat LLM *openai/gpt-oss-120b* via the free tier of Groq, which has daily limits that are practically very difficult to reach within the scope of the workshop. If someone does reach the daily limit for this model, I will suggest another model from the Groq free tier that will also perform well.

Technology stack

- Java Spring Boot
- REST, port 8082
- Spring AI
- MCP Client
- pgvector vector database
- Open Telemetry + Grafana
- Docker infrastructure

The embedding model will be OpenAI's paid *text-embedding-3-small*.

This embedding model is paid but we chose it having in mind its high quality. A pre-generated OpenAI API key is provided by the instructor and everyone can use it.

Expected output

- REST endpoint for direct LLM communication
- REST endpoint for Naive RAG queries
- REST endpoint for structured Naive RAG responses as typed objects
- REST endpoint for Agentic RAG queries working in local or distributed mode via REST client or by communicating with MCP server

Development steps for our Agentic RAG

We will divide the development process into several steps. Each step will be isolated in a separate Git branch, giving us incrementally built working versions of the project.

```
workshop-v1-initial  
workshop-v2-llm  
workshop-v3-knowledge-base  
workshop-v4-naive-rag  
workshop-v5-agentic-rag  
workshop-v6-agentic-rag-rest-client  
workshop-v7-agentic-rag-mcp-client
```

Step 1: workshop-v1-initial

A pre-prepared multi-module project structure is available, consisting of five modules (Java projects). As mentioned, the projects video-production-store, video-production-store-client, and video-production-store-mcp-server are already complete - we will not modify them, only use them.

The base structure of the rag-ingestor and rag-retriever projects is ready as defined sub-projects within the overall project.

Observability is configured for both projects, allowing us to monitor the execution flow of operations when we invoke RAG functionality.

Grafana Observability Dashboard: <http://localhost:3000>

Observability is implemented via OpenTelemetry and Grafana. It allows us to visually trace the entire execution path of each request - from the moment it is received, through calls to the LLM and data sources, to the formation of the final response. This will be especially useful in the Agentic RAG steps, where the agent makes autonomous decisions and calls multiple data sources repeatedly. Observability will show us exactly what is happening "under the hood".

The functionality provided by the video-production-store project, which we will integrate in later steps, can be reviewed at:

<http://localhost:8080/video-production-store/swagger-ui/index.html>

Step 2: workshop-v2-llm

First, we will implement direct communication with the LLM to verify that:

- the selected model has no relevant data about the information we are interested in
- if it does have some knowledge on the topic, it is limited

Based on its responses, it will become clear how the LLM hallucinates.

This functionality will be introduced in the rag-retriever project.

GET <http://localhost:8082/rag-retriever/llm/ask?prompt=What is Movie Machine Pro?>

A document with prepared sample requests to the application endpoints is available.

Step 3: workshop-v3-knowledge-base

In this step we will bring our Knowledge Base into operation, which will subsequently serve the RAG. Development will take place in the rag-ingestor project. For the vector database we will use pgvector via a Docker container. The embedding model we will use is OpenAI's paid *text-embedding-3-small*. An API key will be provided.

We will create functionality for reloading the vector database.

POST <http://localhost:8081/rag-ingestor/knowledge-base/reload>

Here we will focus on the ETL steps during data ingestion, as well as on strategies for writing data to a vector database and which parameters influence how the data is stored and how the subsequent search - process executed as a subprocess within the RAG functionality will operate.

We will create functionality for searching our Knowledge Base.

GET <http://localhost:8081/rag-ingestor/knowledge-base/search?query=What is Movie Machine Pro?>

We have a list of pre-prepared queries for testing the loading and searching.

With this, the Ingestion phase of our RAG is complete. We have loaded our data into our own Knowledge Base.

Step 4: workshop-v4-naive-rag

In this step we will create a RAG in which the LLM will decide, based on the data in its own Knowledge Base, how to gather and combine information to construct answers to submitted queries.

We will work primarily in the rag-retriever project. The vector database is already loaded with our documents. The LLM that will decide how to form the answers will be *openai/gpt-oss-120b* provided via the Groq free tier.

We will create an endpoint for asking the LLM which will ask our Knowledge Base:

GET <http://localhost:8082/rag-retriever/naive-rag/ask?prompt=What is Media 100?>

We have a prepared list of questions to ask the RAG. The answers of some questions are present in our documents loaded into the Knowledge Base, while others are not.

The result of the submitted questions can be a text response formed by the LLM, or a pre-defined typed object that can subsequently be used in business logic.

GET <http://localhost:8082/rag-retriever/naive-rag/video-editing-cards>

Here we will examine which factors affect the quality of the responses:

- how data is distributed across the documents
- how data splitting before being stored in the vector database is a factor
- which parameters can affect the responses we receive
- we will encounter technical limitations and I will mention solutions

We will note that for not all use cases a straightforward approach is a great solution.

Step 5: workshop-v5-agentic-rag

In this step we will evolve our Naive RAG into a local running Agentic RAG.

We will have two data sources:

- our Knowledge Base
- an InMemory simulation of the relational database corresponding to the database of the video-production-store application

The LLM of our Agentic RAG will be able to autonomously decide, based on the submitted questions:

- which data source to retrieve information from
- how to assemble and structure the information
- how many times to call the various data sources while formulating the result

We will create an endpoint for asking the Agent RAG:

```
GET http://localhost:8082/rag-retriever/agentic-rag/ask?prompt=Is
386 processor enough for a computer for working with Movie Machine
Pro?
```

The operation flow will be examined using the observability tools.

Key areas of focus:

- tracing the entire process of query execution, information gathering, and response delivery
- how using a different LLM affects the quality of the response
- examining cases where changing the strategy can produce different results
- how different technical parameters can affect the result
- how the nature of our own data can affect the result
- it will become clear that RAG cannot always solve a problem easily and in a straightforward manner

Step 6: workshop-v6-agentic-rag-rest-client

In this step we will evolve our Agentic RAG to operate in a distributed environment.

We will integrate with our video-production-store application using the pre-built REST client library from the video-production-store-client project.

Again, we will have two data sources:

- our Knowledge Base
- the real relational database of the video-production-store application

We will examine how the operation flow proceeds in distributed systems. Using the observability tools in particular, how a single user query triggers a chain of calls across multiple services, and how the distributed traces in Grafana allow us to follow the full execution path end-to-end - from the initial request through the agent's tool invocations to the final response.

Step 7: workshop-v7-agentic-rag-mcp-client

In this step we will evolve our Agentic RAG, which already operates at the distributed level, to use a next level of independent isolation and abstraction - integration with an MCP Server.

We will integrate with the pre-built video-production-store-mcp-server.

The video-production-store-mcp-server application in turn accesses the real database of the core video-production-store application internally via the video-production-store-client REST client library. This detail is irrelevant to us, because we treat it as a black-box MCP Server.

Again, we will have two data sources:

- our Knowledge Base
- the real relational database of the video-production-store application

We will examine how the operation flow proceeds in distributed software applications connected via the MCP protocol. In particular, how the agent discovers and invokes MCP tools, and how this differs from the direct REST client approach used in the previous step.

Application endpoints and useful URLs

video-production-store

<http://localhost:8080/video-production-store/swagger-ui.html>

video-production-store-mcp-server

`npx @modelcontextprotocol/inspector http://localhost:8083`

rag-ingestor

POST <http://localhost:8081/rag-ingestor/knowledge-base/reload>

GET <http://localhost:8081/rag-ingestor/knowledge-base/search?query=What is Movie Machine Pro?>

rag-retriever

Grafana Dashboard <http://localhost:3000>

GET <http://localhost:8082/rag-retriever/llm/ask?prompt=What is Movie Machine Pro?>

GET <http://localhost:8082/rag-retriever/naive-rag/ask?prompt=What is Media 100?>

GET <http://localhost:8082/rag-retriever/naive-rag/video-editing-cards>

GET <http://localhost:8082/rag-retriever/agentic-rag/ask?prompt=Is 386 processor enough for a computer for working with Movie Machine Pro?>